BẢN TỔNG HỢP VÀ PHÂN TÍCH CHUYÊN SÂU PAPER
Self-Refine: Iterative Refinement with Self-Feedback

Tác giả: Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al.

Carnegie Mellon University, Allen Institute for AI & Google Research — Hội nghị **NeurIPS 2023**

Mã định danh: arXiv:2303.17651 | Mã nguồn: https://github.com/madaan/self-refine

---

1. Bối Cảnh & Điểm Giới Hạn Của Mô Hình Sinh Một Lượt (One-Step Generation)

Trong cuộc sống hàng ngày, con người khi viết văn, thiết kế kiến trúc hay lập trình phần mềm **không bao giờ tạo ra sản phẩm hoàn hảo ngay từ bản nháp đầu tiên**. Một kỹ sư giỏi luôn viết bản phác thảo → đọc lại tự phê bình → phát hiện lỗi → chỉnh sửa lặp đi lặp lại cho đến khi đạt chuẩn.

Tuy nhiên, trước công trình Self-Refine (2023), hầu hết các hệ thống AI ứng dụng LLM (ngay cả kỹ thuật Chain-of-Thought) đều vận hành theo mô hình **Sinh Một Lượt (One-Step Feedforward Generation)**:

$$y \sim P(y \mid x) \tag{1}$$

Nếu câu trả lời ban đầu có một lỗi sai nhỏ (về logic, độ tối ưu thuật toán, hoặc định dạng), hệ thống lập tức xuất ra màn hình cho người dùng mà không có bất kỳ cơ hội tự sửa đổi nào. Các giải pháp truyền thống trước đó đòi hỏi phải huấn luyện thêm mô hình thưởng (Reward Model trong RLHF) hoặc thu thập hàng chục nghìn cặp dữ liệu sửa sai $(y_{\text{sai}}, y_{\text{đúng}})$ rất tốn kém.

LÝ THUYẾT CỐT LÕI: Tư Tưởng Đột Phá Của Self-Refine

Self-Refine đặt ra câu hỏi mang tính cách mạng: *"Tại sao chúng ta không tận dụng chính năng lực ngôn ngữ của LLM để nó tự đóng vai Người Phản Biện (Reviewer) và tự hoàn thiện câu trả lời của chính nó mà không cần bất kỳ dữ liệu huấn luyện bổ sung nào?"*

---

2. Mô Hình Hóa Toán Học & Vòng Lặp Tuần Hoàn 3 Pha

Self-Refine sử dụng duy nhất một mô hình ngôn ngữ lớn M (như GPT-3.5, ChatGPT hoặc GPT-4) đóng luân phiên 3 vai trò kiến trúc trong chu trình lặp:

MÔ HÌNH HÓA TOÁN HỌC: Quy Trình 3 Bước Tuần Hoàn Của Self-Refine

1. **1. Pha Khởi Tạo Ban Đầu (Generation):**
   Cho câu hỏi/yêu cầu x, mô hình sinh ra bản thảo đầu tiên y_0:
   $$y_0 = M_{\text{gen}}(x) \tag{2}$$
2. **2. Pha Tự Phản Biện (Self-Feedback):**
   Mô hình nhận vào cả đầu vào x và bản thảo hiện tại y_t. Bằng một prompt phản biện chuyên

biệt, mô hình đóng vai một người giám sát nghiêm khắc để chỉ ra các lỗi cụ thể:

$$fb_t = M_{\text{feedback}}(x, y_t) \tag{3}$$

Phản hồi fb_t bắt buộc phải mang tính **Hành Động Hóa (Actionable)**: chỉ rõ lỗi ở dòng nào, tại sao sai và gợi ý hướng khắc phục.

3. **3. Pha Tinh Chỉnh (Refinement):**
Mô hình nhận vào bộ ba: câu hỏi gốc x, bản thảo cũ y_t, và bản phê bình fb_t để sinh ra phiên bản nâng cấp y_{t+1}:
$$y_{t+1} = M_{\text{refine}}(x, y_t, fb_t) \tag{4}$$

Vòng lặp tiếp tục cho đến khi thỏa mãn điều kiện dừng: hoặc bản phản biện fb_t xác nhận không còn lỗi cần sửa (Stop signal), hoặc đạt số vòng lặp tối đa $T_{\max}$ (thường chọn $T_{\max} = 3 - 4$).

—

3. Bốn Tiêu Chuẩn Của Bản Phản Biện Chất Lượng (Quality Feedback)

Madaan et al. chỉ ra rằng thành công của Self-Refine phụ thuộc 100% vào chất lượng của pha Feedback. Một câu phản hồi chung chung (như *"Câu trả lời chưa tốt, hãy sửa lại"*) sẽ hoàn toàn vô dụng. Phản biện chuẩn phải thỏa mãn 4 tiêu chuẩn:

1. **Tính hành động (Actionable):** Đưa ra chỉ dẫn sửa đổi rõ ràng, có thể thực thi được bằng code hoặc câu từ cụ thể.
2. **Tính cụ thể (Specific):** Trích dẫn chính xác dòng code, biến số hoặc luận điểm gặp vấn đề.
3. **Tính phân loại có cấu trúc (Structured Rubric):** Đánh giá theo các tiêu chí xác định trước (tính đúng đắn, tính ngắn gọn, độ phức tạp thời gian).
4. **Không phá vỡ phần đúng (Preservation):** Chỉ yêu cầu sửa phần lỗi, giữ nguyên toàn bộ các phần đã làm tốt của bản thảo trước.

—

4. Kết Quả Thực Nghiệm Đột Phá Trên 7 Bài Toán Đa Dạng

Bài báo đánh giá Self-Refine trên 7 miền bài toán rộng lớn từ ngôn ngữ, mã nguồn đến logic toán học:

Miền Tác Vụ	Tiêu Chí Đánh Giá	Mức Cải Thiện Của Self-Refine
Code Optimization	Thời gian thực thi mã nguồn Python	Giảm **22.4%** thời gian chạy code, tối ưu độ phức tạp
Code Readability	Chuẩn cấu trúc PEP8, tài liệu hóa docstring	Tăng điểm đánh giá của chuyên gia người **+1.4 / 5.0**
Math Reasoning	Lập luận toán học GSM8K, SVAMP	Tự phát hiện bước biến đổi nhầm, tăng độ chính xác
Dialog Response	Mức độ an toàn, tự nhiên và thấu cảm	Tăng tỷ lệ ưu tiên của người đánh giá lên **68.3%**
Constrained Generation	Tuân thủ danh sách từ khóa bắt buộc	Tăng tỷ lệ thỏa mãn ràng buộc từ **54% lên 88%**

THỰC NGHIỆM ĐỐI CHỨNG: Con Số Then Chốt Từ Bài Báo

- Trên toàn bộ 7 bài toán, Self-Refine giúp các mô hình (ngay cả mô hình mạnh nhất thế giới như GPT-4) cải thiện trung bình tuyệt đối **20% hiệu năng** so với việc sinh 1 lượt thông thường.
- Đa số các cải thiện vượt bậc diễn ra ngay ở **vòng lặp thứ 1 và thứ 2**; từ vòng lặp thứ 4 trở đi, hiệu năng đạt trạng thái bão hòa (Plateau).

---

5. Giới Hạn Lý Thuyết: Cảnh Báo "Sửa Đúng Thành Sai" (Degradation)

Một phát hiện quan trọng mà các nghiên cứu tiếp nối (đặc biệt là Huang et al., 2023) đã chỉ ra:

LÝ THUYẾT CỐT LÕI: Điểm Yếu Khi Chỉ Tự Phản Biện Nội Tại (Pure Self-Reflection)

Nếu LLM chỉ "tự ngẫm nghĩ một mình" mà **không có phản hồi khách quan từ môi trường thực tế** (như không có trình biên dịch chạy thử code, không có CSDL kiểm tra), mô hình có thể gặp hiện tượng:

- **Tự tin thái quá vào cái sai (Hallucinated Feedback):** Tự nghĩ ra một lý do sai để bắt mình sửa theo.
- **Sửa đúng thành sai (De-refinement):** Bản thảo đầu tiên đúng, nhưng qua 2 vòng phản biện không có căn cứ lại biến thành câu trả lời sai.

Bài học kỹ sư: Trong môi trường sản xuất, Self-Refine bắt buộc phải kết hợp với **Tín Hiệu Kiểm Thử Khách Quan (Ground-Truth External Signals)**.

---

6. Ánh Xạ Trực Tiếp Vào Dự Án Office Asset Agent

Trong hệ thống `office-asset-agent`, nguyên lý Self-Refine kết hợp tín hiệu ngoại vi được hiện thực hóa tại nút xử lý lỗi trong `src/agents/orchestrator.py` (Tuần 10):

```
# SELF-REFINE KET HOP TIN HIEU KHACH QUAN TU SQLITE:
def handle_agent_error_node(state: AgentState) -> AgentState:
    attempt = state.get("error_count", 0) + 1
    state["error_count"] = attempt

    # 1. TIN HIEU KHACH QUAN (Ground-truth feedback tu Database):
    last_err = state.get("tool_output", {}).get("error_message")

    # 2. GENERATE ACTIONABLE FEEDBACK:
    if "no such column" in str(last_err).lower():
        feedback = "Loi: Ban da su dung sai ten cot. Hay kiem tra lai schema bang assets."
    elif "empty result" in str(last_err).lower():
        feedback = "Canh bao: Khong tim thay thiet bi. Hay su dung fuzzy matching hoac Alias."
    else:
        feedback = f"Su co ngoai le: {last_err}. Hay dieu chinh tham so truy van."
```

```
    # 3. REFINEMENT (Agent con doc Feedback va thu lai voi tham so moi):
    state["correction_guidance"] = feedback
    return state
```

—

7. Bộ Câu Hỏi Phản Biện Chuyên Sâu Của Hội Đồng (Defense Insights)

ỨNG DỤNG VÀ PHẢN BIỆN: 3 Câu Hỏi Đắt Giá Nhất Về Self-Refine Mà Hội Đồng Sẽ Hỏi Bạn

1. **Hội đồng hỏi:** *"Tại sao Self-Refine không cần dữ liệu huấn luyện bổ sung mà vẫn cải thiện được 20% chất lượng câu trả lời?"*
   Đáp án chuẩn: Vì việc giải một bài toán (Generation) và việc kiểm tra một lời giải (Verification) đòi hỏi hai mức độ phức tạp nhận thức khác nhau (tương tự định lý P vs NP). Khi sinh câu trả lời trong một lượt, mô hình bị giới hạn bởi luồng token tuyến tính. Khi tách riêng thành pha Feedback, toàn bộ năng lực chú ý (Attention) của mô hình được dồn vào việc rà soát từng dòng, giúp nó nhận ra những lỗi ngớ ngẩn mà lúc đang sinh câu trả lời nó đã vô tình bỏ qua.

2. **Hội đồng hỏi:** *"Làm sao em ngăn chặn hiện tượng vòng lặp vô tận (Infinite Loop) hoặc mô hình sửa đúng thành sai?"*
   Đáp án chuẩn: Em áp dụng 2 chốt chặn cứng trong LangGraph: (1) **Ngưỡng thử lại tối đa (Max Iterations = 3)** — nếu sau 3 lần tinh chỉnh vẫn không đạt, hệ thống ngắt mạch (Circuit Breaker) và báo cáo cho quản trị viên; (2) Không dựa vào cảm tính nội tại của LLM mà dùng **Mã lỗi khách quan từ SQLite** làm căn cứ phản biện.

3. **Hội đồng hỏi:** *"Chi phí token khi áp dụng Self-Refine tăng lên bao nhiêu lần? Có xứng đáng không?"*
   Đáp án chuẩn: Trung bình mỗi vòng lặp tốn thêm khoảng 1.5 đến 2 lần số token của lượt đầu tiên. Tuy nhiên, trong đồ án của em, Self-Refine **không chạy tràn lan trên mọi câu chat**, mà chỉ được kích hoạt theo điều kiện (Conditional Edge) khi và chỉ khi Tool ngoại vi báo lỗi. Do đó, 90% các câu hỏi thông thường chỉ tốn 1 lượt sinh, và chi phí bổ sung cho các trường hợp lỗi là hoàn toàn xứng đáng để đổi lấy sự ổn định tuyệt đối của hệ thống.